Appunti di Calcolo Numerico

2 marzo 2026

Indice

1 Introduzione **2**

2 Teoria della Calcolabilità **3**

2.1 Fondamenti della Calcolabilità 3

2.1.1 Enumerabilità, computabilità e decidibilità 3

2.2 Funzioni Ricorsive 6

2.2.1 Funzioni ricorsive primitive 6

1 Introduzione

L'Informatica Teorica si divide in due macro-aree fondamentali:

- **Teoria della computabilità**: cerca modelli formali che si avvicinino al concetto logico e matematico di ALGORITMO.
- **Teoria della complessità computazionale**: si occupa di determinare la quantità minima di risorse (principalmente spazio e tempo) necessarie ad un algoritmo per risolvere un dato problema.

Hai perfettamente ragione, colpa mia! Un documento matematico impaginato in quel modo risulta pesante da leggere, e i simboli "incollati" alle lettere (come $\exists\mathcal{M}$) sono un pugno nell'occhio.Per risolvere il problema, ho fatto due cose:Ho inserito un comando prima di ogni blocco importante (Definizione,

Proposizione, Osservazione) per dare "respiro" al testo ed evitare l'effetto muro.Ho aggiunto uno spazio esplicito (usando in LaTeX) dopo i quantificatori, in modo che ora tu abbia cose pulite come $\exists\ \mathcal{M}$ e $\forall\ w$.Ecco la versione corretta e impaginata molto meglio:Snippet di codice

2 Teoria della Calcolabilità

2.1 Fondamenti della Calcolabilità

2.1.1 Enumerabilità, computabilità e decidibilità

Cos'è un algoritmo? È una NOZIONE PRIMITIVA $\longrightarrow$ non ha una definizione formale, ma lo abbiamo nella nostra testa e sappiamo riconoscerlo.

Definizioni formali
Per prima cosa ci mettiamo in un <u>contesto formale</u>:

- Σ : alfabeto
- Σ^* : stringhe finite su Σ
- $\mathcal{L} \subseteq \Sigma^*$: linguaggio

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice **DECIDIBILE** quando $\exists\ \mathcal{M}$ algoritmo che, data $w \in \Sigma^*$, mi dice in tempo finito se $w \in \mathcal{L}$ oppure $w \notin \mathcal{L}$.

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice **ENUMERABILE** quando $\exists\ \mathcal{M}$ algoritmo che, fatto partire su input vuoto, <u>produce la lista delle stringhe di $\mathcal{L}$</u> (ciascuna stringa in tempo finito).

Definizione: Siano Σ_1, Σ_2 alfabeti, $\mathcal{L}_1 \subseteq \Sigma_1^*$, $\mathcal{L}_2 \subseteq \Sigma_2^*$.
$f : \mathcal{L}_1 \to \mathcal{L}_2$ si dice **COMPUTABILE** quando $\exists\ \mathcal{M}$ algoritmo che, fatto partire su input $w \in \mathcal{L}_1$, restituisce $f(w) \in \mathcal{L}_2$ in tempo finito.

Proposizione: Sia Σ un alfabeto.

$$\mathcal{L} \text{ DECIDIBILE} \implies \mathcal{L} \text{ ENUMERABILE}$$

Dimostrazione:
Sia $\mathcal{M}$ un algoritmo di decisione per $\mathcal{L}$. Descrivo un algoritmo di enumerazione per $\mathcal{L}$: $\forall$ stringa $w \in \Sigma^*$, eseguo $\mathcal{M}$ su w:

- Se $w \in \mathcal{L}$, aggiungo w alla lista.
- Se $w \notin \mathcal{L}$, non l'aggiungo.

Osservazione: Σ numerabile $\implies$ Σ^* enumerabile.
Sia $\Sigma = \{a_1, a_2, \dots, a_n, \dots\}$.
Se cerchiamo di enumerare le stringhe di lunghezza 2 su Σ tramite un PROCEDIMENTO DIAGONALE DI CANTOR:

$$
\begin{array}{c|ccccccc}
 & a_1 & & a_2 & & a_3 & & \dots \\
\hline
a_1 & a_1a_1 & \rightarrow & a_1a_2 & & a_1a_3 & \rightarrow & \dots \\
 & & \swarrow & & \nearrow & & \swarrow & \\
a_2 & a_2a_1 & & a_2a_2 & & a_2a_3 & & \dots \\
 & \downarrow & \nearrow & & \swarrow & & & \\
a_3 & a_3a_1 & & a_3a_2 & & \dots & & \\
 & & \swarrow & & & & & \\
\vdots & \vdots & & \vdots & & & & \ddots
\end{array}
$$

Usando il procedimento diagonale è possibile enumerare le parole e poi trovare un algoritmo di enumerazione per le stringhe.

Attenzione: $\mathcal{L}$ decidibile $\Longleftarrow$ $\mathcal{L}$ enumerabile (NON vale il viceversa in generale).
MA vale la seguente...

Proposizione:

$$\mathcal{L} \text{ DECIDIBILE} \iff \mathcal{L} \text{ ENUMERABILE e } \mathcal{L}^c \text{ ENUMERABILE} \quad (\text{dove } \mathcal{L}^c = \Sigma^* \setminus \mathcal{L})$$

Dimostrazione:

$\Longrightarrow$) $\mathcal{L}$ decidibile $\Longrightarrow$ $\mathcal{L}$ enumerabile (già dimostrato, OK).
$\mathcal{L}$ decidibile $\Longrightarrow$ $\mathcal{L}^c$ enumerabile: analogo.

$\Longleftarrow$) Sia $\mathcal{M}$ algoritmo di enumerazione per $\mathcal{L}$: $w_1, w_2, w_3, \dots, w_n, \dots$
Sia $\mathcal{M}^c$ algoritmo di enumerazione per $\mathcal{L}^c$: $v_1, v_2, v_3, \dots, v_n, \dots$
Costruisco un algoritmo di decisione per $\mathcal{L}$:
Su input $w \in \Sigma^*$, eseguo $\mathcal{M}$ e $\mathcal{M}^c$ in maniera ALTERNATA. Ogni volta che viene prodotta una stringa X, la confronto con w:

- Se $X \neq w$, proseguo.
- Se $X = w$ e X è prodotta da $\mathcal{M}$, accetto w.
- Se $X = w$ e X è prodotta da $\mathcal{M}^c$, rifiuto w.

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice **SEMIDECIDIBILE** quando $\exists$ $\mathcal{M}$ algoritmo che, fatto partire su input w:

- Se $w \in \mathcal{L}$, termina e accetta w.
- Se $w \notin \mathcal{L}$, può non terminare (va in loop).

Osservazione: $\mathcal{L}$ semidecidibile $\iff$ $\exists$ $\mathcal{M}$ algoritmo che, su input w:

- Se $w \in \mathcal{L}$, termina e accetta w.
- Se $w \notin \mathcal{L}$, non termina.

Proposizione:

$$\mathcal{L} \text{ ENUMERABILE} \iff \mathcal{L} \text{ SEMIDECIDIBILE}$$

Dimostrazione:

$\Longrightarrow$) Sia $\mathcal{M}$ algoritmo di enumerazione per $\mathcal{L}$: $w_1, w_2, w_3, \dots$
Costruisco un algoritmo di semidecisione per $\mathcal{L}$:
Data $w \in \Sigma^*$, confronto w con le stringhe prodotte via via da $\mathcal{M}$. Se w è tra queste, termino e accetto w.

$\Longleftarrow$) Sia $\mathcal{M}$ algoritmo di semidecisione per $\mathcal{L}$ (che non termina su tutte le stringhe del complemento). Eseguo i passi di computazione intrecciandoli (dovetailing):

$$
\begin{array}{r|cccccccc}
w_1: & x & & x & & x & \dots & x \\
 & & \swarrow & & \nearrow & & \swarrow & \\
w_2: & x & & x & & x & \dots & \circ \\
 & \downarrow & \nearrow & & \swarrow & & & \\
w_3: & x & & x & & \dots & & \\
 & & \swarrow & & & & & \\
w_4: & x & & \dots & & & & \\
\vdots & \vdots & & & & & &
\end{array}
$$

(Eseguo 1 passo su w_1; poi 1 passo su w_1 e 1 su w_2; poi 1 passo su w_1, w_2 e w_3, ecc. Quando una computazione termina e accetta (es. in w_2), metto la stringa corrispondente nella lista).

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice avere **FUNZIONE CARATTERISTICA** $\chi_{\mathcal{L}}$ se la funzione è così definita:

$$\chi_{\mathcal{L}} : \Sigma^* \to \{0,1\} \quad : \quad w \mapsto \begin{cases} 0, & w \notin \mathcal{L} \\ 1, & w \in \mathcal{L} \end{cases}$$

Proposizione:

$$\mathcal{L} \text{ DECIDIBILE} \iff \chi_{\mathcal{L}} \text{ COMPUTABILE}$$

Dimostrazione:

$\Longrightarrow$) Sia $\mathcal{M}$ algoritmo di decisione per $\mathcal{L}$. Costruisco un algoritmo di calcolo per $\chi_{\mathcal{L}}$:
Data $w \in \Sigma^*$, eseguo $\mathcal{M}$ su w:

- Se $w \in \mathcal{L}$, scrivo 1.
- Se $w \notin \mathcal{L}$, scrivo 0.

$\Longleftarrow$) Sia $\mathcal{M}$ algoritmo di calcolo per $\chi_{\mathcal{L}}$. Costruisco un algoritmo di decisione per $\mathcal{L}$:
Su input w, calcolo $\chi_{\mathcal{L}}(w)$ (usando $\mathcal{M}$):

- Se $\chi_{\mathcal{L}}(w) = 1$, $w \in \mathcal{L}$.
- Se $\chi_{\mathcal{L}}(w) = 0$, $w \notin \mathcal{L}$.

Proposizione: Siano $\mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^*$ DECIDIBILI. Allora:

1. $\mathcal{L}_1 \cup \mathcal{L}_2$ è decidibile
2. $\mathcal{L}_1 \cap \mathcal{L}_2$ è decidibile
3. $\mathcal{L}_1^c$ è decidibile

Dimostrazione:

1. **Algoritmo di decisione per $\mathcal{L}_1 \cup \mathcal{L}_2$:**
   Su input w: Eseguo $\mathcal{M}_1$. Se $w \in \mathcal{L}_1$ termino e accetto.
   Se $w \notin \mathcal{L}_1$, eseguo $\mathcal{M}_2$:
   - Se $w \in \mathcal{L}_2$, termino e accetto.
   - Altrimenti rifiuto.
2. **Algoritmo di decisione per $\mathcal{L}_1 \cap \mathcal{L}_2$:**
   Su input w: Eseguo $\mathcal{M}_1$:
   - Se $w \notin \mathcal{L}_1$, rifiuto.

- Se $w \in \mathcal{L}_1$, osservo $\mathcal{M}_2$:
  - Se $w \notin \mathcal{L}_2$, rifiuto.
  - Altrimenti accetto.

3. **Algoritmo di decisione per $\mathcal{L}_1^c$:**
   Su input w, eseguo $\mathcal{M}_1$ e SCAMBIO LE SUE RISPOSTE (cioè l'algoritmo di decisione inverte l'output).

2.2 Funzioni Ricorsive

2.2.1 Funzioni ricorsive primitive

Ci concentriamo su funzioni del tipo:

$$f : \mathbb{N}^k \to \mathbb{N}$$
$$(x_1, \dots, x_k) \in \mathbb{N}^k$$
$$f(x_1, \dots, x_k)$$

Indicheremo il vettore di input come $\vec{x} = (x_1, \dots, x_k)$.

FUNZIONI INIZIALI

1. **Costanti k-arie di valore 0**:
$$C_0^{(k)} : \mathbb{N}^k \to \mathbb{N}$$
$$C_0^{(k)}(\vec{x}) = 0$$

2. **Funzione successore**:
$$S : \mathbb{N} \to \mathbb{N}$$
$$S(x) = x + 1$$

3. **Proiezioni j-esime k-arie**:
$$\mathcal{E}_j^{(k)} : \mathbb{N}^k \to \mathbb{N}$$
$$\mathcal{E}_j^{(k)}(\vec{x}) = x_j \quad \text{(estrae l'argomento j-esimo)}$$

Definizione: Sia A un insieme, Σ un alfabeto.
Una **Codifica** è una funzione $p : A \to \Sigma^*$ (da A ottengo una stringa).

Definizione: Una **Rappresentazione** (Buona codifica) di A in Σ si ha quando:

1. p è **computabile** ($\exists$ algoritmo che per $\forall\ a \in A$ mi calcola $p(a)$).
2. p è **iniettiva** (la codifica è unica).
3. $p(A)$ è **decidibile** e p è **effettivamente invertibile**, cioè: $\exists$ un algoritmo che, $\forall\ w \in p(A)$, mi restituisce $a \in A$ tale che $p(a) = w$ (Posso ricostruire senza ambiguità l'elemento di A dalla codifica).

Proposizione: Sia Σ un alfabeto. Allora $\exists\ p$ rappresentazione in $\{0,1\}^*$.
$p : \Sigma^* \to \{0,1\}^*$ (Posso codificare in binario).
Dimostrazione:
Sia $\Sigma = \{a_1, a_2, \dots, a_n, \dots\}$.
$p : \Sigma^* \to \{0,1\}^*$
$p(a_{i_1} a_{i_2} \dots a_{i_n}) = 01 \dots 101 \dots 1 \dots 01 \dots 1$ (dove i blocchi di 1 dipendono dagli indici i_k).
p è una rappresentazione di Σ^* in $\{0,1\}^*$.

Codifica unaria per numeri naturali:
$\Sigma = \{1\}$

$0 \mapsto 1$
$1 \mapsto 11$
$2 \mapsto 111$
$n \mapsto \underbrace{11\ldots1}_{n+1 \text{ volte}}$

Osservazione: Le funzioni iniziali sono computabili.
Esempio: $S(\underbrace{11\ldots1}_{n+1}) = \underbrace{1\ldots1}_{n+2}$

Definizione: Composizione generalizzata
Date $g_1 \ldots g_n : \mathbb{N}^k \to \mathbb{N}$ e $f : \mathbb{N}^n \to \mathbb{N}$.
La **composizione generalizzata** (o sostituzione) di f con $(g_1, \ldots, g_n)$ è la funzione:

$$f \circ (g_1, \ldots, g_n) : \mathbb{N}^k \to \mathbb{N}$$

$$(f \circ (g_1, \ldots, g_n))(\vec{x}) = f(g_1(\vec{x}), \ldots, g_n(\vec{x}))$$

Proposizione: $f, g_1, \ldots, g_n$ computabili $\implies f \circ (g_1, \ldots, g_n)$ computabile.
Dimostrazione:
Siano $F, G_1, \ldots, G_n$ algoritmo di calcolo per $f, g_1, \ldots, g_n$.
Algoritmo di calcolo per $f \circ (g_1, \ldots, g_n)$:
Data $\vec{x} \in \mathbb{N}^k$:

- Uso $G_1, \ldots, G_n$ per calcolare $g_1(\vec{x}), \ldots, g_n(\vec{x})$.
- Uso F per calcolare $f(g_1(\vec{x}), \ldots, g_n(\vec{x}))$.

Definizione: Ricorsione Primitiva (RP)
Date $g : \mathbb{N}^k \to \mathbb{N}$, $\varphi : \mathbb{N}^{k+2} \to \mathbb{N}$.
Diciamo che $f : \mathbb{N}^{k+1} \to \mathbb{N}$ è definita per **ricorsione primitiva** (RP) da g e φ quando:

- $f(\vec{x}, 0) = g(\vec{x})$ *(caso base)*
- $f(\vec{x}, y+1) = \varphi(\vec{x}, y, f(\vec{x}, y))$ *(passo induttivo)*

Proposizione: g, φ computabili $\implies f$ definita per RP da g, φ è computabile.
Dimostrazione:
Descrivo un algoritmo per calcolare f, ragionando per induzione sull'ultimo parametro di f. In pratica facciamo vedere che fissato $\vec{x} \in \mathbb{N}^k$, per ogni $y \in \mathbb{N}$, so calcolare $f(\vec{x}, y)$:

- (Base $y = 0$): $f(\vec{x}, 0) = g(\vec{x})$, e g è computabile.
- (Iterazione): Supponiamo di saper calcolare $f(\vec{x}, y)$.
  $f(\vec{x}, y+1) = \varphi(\vec{x}, y, f(\vec{x}, y))$. Essendo φ computabile (per hp) e $f(\vec{x}, y)$ noto, anche il calcolo al passo successivo restituisce un risultato in tempo finito.

Definizione: Derivazione Ricorsiva Primitiva (drp)
È una sequenza di funzioni $f_1, f_2, \ldots, f_n$ t.c. $\forall\ i = 1, \ldots, n$:

- f_i è **funzione iniziale**, oppure
- f_i è **ottenuta da** f_j, f_k **per RP** con $j, k < i$, oppure
- f_i è **ottenuta per composizione generalizzata** da funzioni precedenti

Definizione: Funzione Ricorsiva Primitiva (rp)
Una **funzione ricorsiva primitiva (rp)** è una funzione che compare in coda ad una drp (ovvero è l'ultima funzione di una drp).

Osservazione:

1. Ogni prefisso di una drp è una drp.
2. Ogni funzione che compare in una drp è rp.

Proposizione: Ogni funzione rp è computabile.
Dimostrazione:
Sia f rp. Dimostro per **induzione sulla lunghezza n di una drp** che termina con f:

- **Caso base** ($n = 1$): $f \implies f$ è funzione iniziale, dunque è computabile.
- **Passo induttivo**: Supponiamo che tutte le funzione che compaiono in una drp di lunghezza $< n$ siano computabili.
  Sia $f_1, \ldots, f_n = f$.
  f deriva da funzioni precedenti $f_j, \ldots, f_k$ le quali sono computabili per Hp induttiva. Siccome le operazioni di composizione e RP mantengono la computabilità (come dimostrato in precedenza), anche f è computabile.

Proposizione:

1. $g : \mathbb{N}^k \to \mathbb{N}$, $\varphi : \mathbb{N}^{k+2} \to \mathbb{N}$ rp $\implies f$ ottenuta per RP da g e φ è rp.
2. $\varphi : \mathbb{N}^n \to \mathbb{N}$, $g_1, \ldots, g_n : \mathbb{N}^k \to \mathbb{N}$ rp $\implies f \circ (g_1, \ldots, g_n)$ rp.

Dimostrazione:

1. Siano $g_1, \ldots, g_s, g$ una drp per g, e $\varphi_1, \ldots, \varphi_r, \varphi$ una drp per φ.
   La sequenza $g_1, \ldots, g_s, g, \varphi_1, \ldots, \varphi_r, \varphi, f$ è essa stessa una drp (poiché per ipotesi tutte le precedenti sono drp e f è ottenuta da esse). Di conseguenza, f è rp perché compare in una drp.

ESEMPI DI FUNZIONI RICORSIVE PRIMITIVE

1) Costante m-esima k-aria: $C_m^{(k)} : \mathbb{N}^k \to \mathbb{N}$, dove $C_m^{(k)}(x_1, \ldots, x_k) = m \quad (m \in \mathbb{N})$, con k che indica il numero di argomenti (es. $C_0^{(2)}(x, y) = 0$) e m che indica il valore della costante (es. $C_3^{(2)}(x, y) = 3$). Dimostriamo per induzione su $m \in \mathbb{N}$ che è rp:

- Base: $C_0^{(k)}$ è rp (perché è una **funzione iniziale**).
- Supponiamo che $C_m^{(k)}$ sia rp.
  $C_{m+1}^{(k)}(\vec{x}) = m + 1 = (S \circ C_m^{(k)})(\vec{x})$
  Essendo la composizione tra S (funzione successore, che è iniziale) e $C_m^{(k)}$ (rp per ipotesi induttiva), allora $C_{m+1}^{(k)}$ è rp.

2) Funzione Predecessore: $v : \mathbb{N} \to \mathbb{N}$

$$v(x) = \begin{cases} x - 1, & x > 0 \\ 0, & x = 0 \end{cases}$$

Costruiamo la ricorsione primitiva:

$$\begin{cases} v(0) = 0 \quad \text{(che corrisponde a } C_0^{(0)} \text{ che è rp)} \\ v(x+1) = \left(\mathcal{E}_1^{(2)}\right)(x, v(x)) = x \quad \text{(rp perché } \mathcal{E}_1^{(2)} \text{ è funz. iniziale)} \end{cases}$$

$\implies v(x)$ è rp.

3) Somma: $s : \mathbb{N}^2 \to \mathbb{N}$
$s(x, y) = x + y$

$$\begin{cases} s(x,0) = x = \mathcal{E}_1^{(1)}(x) \quad \text{(rp)} \\ s(x, y+1) = x + y + 1 = \left(S \circ \mathcal{E}_3^{(3)}\right)(x, y, s(x,y)) \quad \text{(rp)} \end{cases}$$

$\implies$ s è rp.

4) Prodotto: $p : \mathbb{N}^2 \to \mathbb{N}$
$p(x, y) = x \cdot y$

$$\begin{cases} p(x,0) = 0 = \left(C_0^{(1)}\right)(x) \quad \text{(rp)} \\ p(x, y+1) = xy + x = \left(s \circ (\mathcal{E}_3^{(3)}, \mathcal{E}_1^{(3)})\right)(x, y, p(x,y)) \quad \text{(rp)} \end{cases}$$

$\implies$ p è rp.

5) Esponenziazione: $\pi : \mathbb{N}^2 \to \mathbb{N}$
$\pi(x, y) = x^y$

$$\begin{cases} \pi(x,0) = x^0 = 1 = C_1^{(1)}(x) \quad \text{(rp)} \\ \pi(x, y+1) = x^y \cdot x = \left(p \circ (\mathcal{E}_3^{(3)}, \mathcal{E}_1^{(3)})\right)(x, y, \pi(x,y)) \quad \text{(rp)} \end{cases}$$

6) Fattoriale: $f : \mathbb{N} \to \mathbb{N}$
$f(x) = x!$

$$\begin{cases} f(0) = 1 \quad \text{(rp)} \\ f(x+1) = (x+1)! = (x+1) \cdot x! = \left(p \circ (S \circ \mathcal{E}_1^{(2)}, \mathcal{E}_2^{(2)})\right)(x, f(x)) \end{cases}$$

7) Differenza Troncata: $\dot{-} : \mathbb{N}^2 \to \mathbb{N}$

$$x \dot{-} y = \begin{cases} x - y, & x \geq y \\ 0, & x < y \end{cases}$$

$$\begin{cases} x \dot{-} 0 = x = \mathcal{E}_1^{(1)}(x) \quad \text{(rp)} \\ x \dot{-} (y+1) = x - y - 1 = \left(v \circ \mathcal{E}_3^{(3)}\right)(x, y, x \dot{-} y) \quad \text{(rp)} \end{cases}$$

8) Distanza: $d : \mathbb{N}^2 \to \mathbb{N}$
$d(x, y) = |x - y|$

$$d(x,y) = (x \dot{-} y) + (y \dot{-} x)$$

(Questo copre entrambi i casi in cui $x \geq y$ o $y > x$)

$$= \left(s \circ (\dot{-} \circ (\mathcal{E}_1^{(2)}, \mathcal{E}_2^{(2)}), \dot{-} \circ (\mathcal{E}_2^{(2)}, \mathcal{E}_1^{(2)}))\right)(x, y)$$

$\implies$ d è rp.

9) Segno: $sg : \mathbb{N} \to \mathbb{N}$

$$sg(x) = \begin{cases} 1, & x > 0 \\ 0, & x = 0 \end{cases}$$

$$\begin{cases} sg(0) = 0 \\ sg(x+1) = 1 = C_1^{(2)}(x, sg(x)) \end{cases}$$

10) Segno Opposto: $\overline{sg} : \mathbb{N} \to \mathbb{N}$

$$\overline{sg}(x) = \begin{cases} 0, & x > 0 \\ 1, & x = 0 \end{cases}$$

Dimostrazione: Possiamo dimostrare che è rp per ricorsione primitiva:

$$\begin{cases} \overline{sg}(0) = 1 = C_1^{(0)} \quad \text{(rp)} \\ \overline{sg}(x+1) = 0 = C_0^{(2)}(x, \overline{sg}(x)) \quad \text{(rp, con composizione di proiezioni se necessario)} \end{cases}$$

$\implies \overline{sg}$ è rp.

Metodo alternativo (per composizione): Possiamo sfruttare la differenza troncata. Sottrarre x a 1 ci restituisce 1 solo se $x = 0$, altrimenti ci restituisce 0:

$$\overline{sg}(x) = 1 \dot{-} x = \left(\dot{-} \circ (C_1^{(1)}, \mathcal{E}_1^{(1)})\right)(x)$$

Essendo ottenuta per composizione di funzioni rp, è rp.

11) Delta di Kronecker: $\delta(x, y)$

$$\delta(x, y) = \begin{cases} 1, & x = y \\ 0, & x \neq y \end{cases}$$

Dimostrazione: Sappiamo che la funzione distanza $d(x, y) = |x - y|$ (che abbiamo già dimostrato essere rp) vale 0 se e solo se $x = y$, ed è strettamente maggiore di 0 altrimenti. Se applichiamo la funzione *Segno Opposto* al risultato della distanza, otteniamo esattamente il comportamento del Delta di Kronecker:

$$\delta(x, y) = \overline{sg}(d(x, y)) = (\overline{sg} \circ d)(x, y)$$

Essendo il Delta di Kronecker ottenuto per composizione di due funzioni rp ($\overline{sg}$ e d), è esso stesso una funzione rp.

Definizione: Operatore di Somma Limitata
Sia $f : \mathbb{N}^{k+1} \to \mathbb{N}$. Definiamo $g : \mathbb{N}^{k+1} \to \mathbb{N}$ ponendo:

$$g(\vec{x}, z) = \sum_{y=0}^{z} f(\vec{x}, y)$$

Diciamo che g è ottenuta per **somma limitata** da f. La funzione che, data f, costruisce g si chiama operatore di somma limitata.

Proposizione: f rp $\implies g$ ottenuta per somma limitata da f è rp.
Dimostrazione:

$$\begin{cases} g(\vec{x}, 0) = \sum_{y=0}^{0} f(\vec{x}, y) = f(\vec{x}, 0) = \left(f \circ (\mathcal{E}_1^{(k)}, \mathcal{E}_2^{(k)}, \ldots, \mathcal{E}_k^{(k)}, C_0^{(k)})\right)(\vec{x}) \\ \\ g(\vec{x}, z+1) = \sum_{y=0}^{z+1} f(\vec{x}, y) = \sum_{y=0}^{z} f(\vec{x}, y) + f(\vec{x}, z+1) = \\ = \left(s \circ (\mathcal{E}_{k+2}^{(k+2)}, f \circ (\mathcal{E}_1^{(k+2)}, \ldots, \mathcal{E}_k^{(k+2)}, S \circ \mathcal{E}_{k+1}^{(k+2)}))\right)(\vec{x}, z, g(\vec{x}, z)) \end{cases}$$

Osservazione:
Data $f : \mathbb{N}^{k+2} \to \mathbb{N}$, possiamo avere $g : \mathbb{N}^{k+1} \to \mathbb{N}$ definita come:

$$g(\vec{x}, z) = \sum_{y=0}^{z} f(\vec{x}, y, z)$$

Definizione: Operatore di Prodotto Limitato
Sia $f : \mathbb{N}^{k+1} \to \mathbb{N}$. Definiamo $g : \mathbb{N}^{k+1} \to \mathbb{N}$ ponendo:

$$g(\vec{x}, z) = \prod_{y=0}^{z} f(\vec{x}, y)$$

Diciamo che g è ottenuta per **prodotto limitato** da f.

Proposizione: f rp $\implies$ g rp.
(La dimostrazione è la stessa della somma limitata, sostituendo l'operatore di somma s con quello di prodotto p).

Esercizio
Sia $g : \mathbb{N}^2 \to \mathbb{N}$ una funzione rp.

a) Sia $f_1(x, y, z_1, \ldots, z_n) = g(x, y)$. Dimostrare che f_1 è rp:

$$f_1(x, y, z_1, \ldots, z_n) = g(x, y) = \left(g \circ (\mathcal{E}_1^{(n+2)}, \mathcal{E}_2^{(n+2)})\right)(x, y, z_1, \ldots, z_n)$$

b) Sia $f_2(x, y) = g(y, x)$. Dimostrare che f_2 sia rp:

$$f_2(x, y) = \left(g \circ (\mathcal{E}_2^{(2)}, \mathcal{E}_1^{(2)})\right)(x, y)$$

c) Sia $f_3(x) = g(x, x)$. Dimostrare che f_3 sia rp:

$$f_3(x) = \left(g \circ (\mathcal{E}_1^{(1)}, \mathcal{E}_1^{(1)})\right)(x)$$